

Diffusion Model

80-minute lecture

Start from the idea of a latent-variable model

We observe data $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ and introduce latent variables $\mathbf{z}$.

A generative model factorizes as

$$p_{\theta}(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}) p_{\theta}(\mathbf{x} | \mathbf{z}), \quad p_{\theta}(\mathbf{x}) = \int p(\mathbf{z}) p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z}.$$

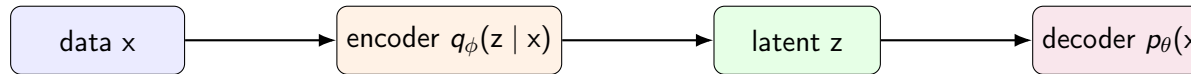
- $p(\mathbf{z})$ is usually simple, for example a standard Gaussian.
- $p_{\theta}(\mathbf{x} | \mathbf{z})$ decodes latent structure into an observation.
- The challenge is that the marginal likelihood $p_{\theta}(\mathbf{x})$ is usually intractable.

Idea: learn a simple latent space where generation is easy, then decode back to data space.

VAE viewpoint: encoder, decoder, variational inference

A VAE introduces an approximate posterior

$$q_{\phi}(z \mid x) \approx p_{\theta}(z \mid x).$$



- The encoder makes posterior inference tractable.
- The decoder learns to reconstruct realistic data from latent codes.
- Training is done by maximizing a lower bound on $\log p_{\theta}(x)$.

Deriving the ELBO

Start from

$$\log p_{\theta}(\mathbf{x}) = \log \int q_{\phi}(\mathbf{z} \mid \mathbf{x}) \frac{p_{\theta}(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z} \mid \mathbf{x})} d\mathbf{z}.$$

Applying Jensen's inequality gives

$$\log p_{\theta}(\mathbf{x}) \geq \mathbb{E}_{q_{\phi}(\mathbf{z} \mid \mathbf{x})} [\log p_{\theta}(\mathbf{x}, \mathbf{z}) - \log q_{\phi}(\mathbf{z} \mid \mathbf{x})] =: \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}).$$

Rearranging,

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_{\phi}(\mathbf{z} \mid \mathbf{x})} [\log p_{\theta}(\mathbf{x} \mid \mathbf{z})] - \text{KL}(q_{\phi}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})).$$

- Reconstruction term: explain the data.
- KL term: keep the approximate posterior close to the prior.

Why bring up the VAE before diffusion?

Because diffusion can be presented as a very structured latent-variable model.

- A VAE uses **one** latent variable z with a learned encoder $q_{\phi}(z | x)$.
- A DDPM uses **many** latent variables $x_1, \dots, x_T$.
- Its forward process $q(x_{1:T} | x_0)$ plays the role of an encoder.
- But this encoder is **fixed by design**, not learned.

Pedagogical slogan

A diffusion model is like a **deep hierarchical VAE** whose inference path is analytically chosen to be Gaussian noising.

DDPM as an extension of the VAE idea

Let the observed data be x_0 and introduce latent variables $x_1, \dots, x_T$.

The **forward process** is fixed:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^T q(x_t \mid x_{t-1}), \quad q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I\right).$$

The **reverse generative model** is learned:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t), \quad p(x_T) = \mathcal{N}(0, I).$$

- Forward chain: gradually destroys information.
- Reverse chain: learns to reconstruct it step by step.
- The terminal latent x_T is simple Gaussian noise.

The DDPM ELBO

Because $q(x_{1:T} \mid x_0)$ is known, we can write a variational lower bound on $\log p_\theta(x_0)$:

$$\log p_\theta(x_0) \geq \mathbb{E}_{q(x_{1:T} \mid x_0)} [\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)] .$$

Expanding the Markov factors gives

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_q \left[\log p(x_T) + \sum_{t=1}^T \log p_\theta(x_{t-1} \mid x_t) - \sum_{t=1}^T \log q(x_t \mid x_{t-1}) \right] .$$

This is exactly an [ELBO](#), just with many latent variables instead of one.

ELBO decomposition into interpretable terms

The standard decomposition is

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_q \left[-\text{KL}(q(\mathbf{x}_T \mid \mathbf{x}_0) \parallel p(\mathbf{x}_T)) - \sum_{t=2}^T \text{KL}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)) + \log p_\theta(\mathbf{x}_0 \mid \mathbf{x}_1) \right].$$

- The last term is a reconstruction term at the final denoising stage.
- The middle terms match the learned reverse transition to the true Gaussian posterior.
- The first term aligns the end of the forward chain with the simple prior.

VAE analogy: one KL in a standard VAE becomes a **sum of local KLs across noise scales**.

Why the simplified DDPM loss becomes noise prediction

The forward marginal has a closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t)I), \quad \mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I).$$

The true posterior $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is Gaussian, so if we parameterize

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t, t), \Sigma_t),$$

then each KL term reduces to a quadratic loss on the mean.

With the standard parameterization, that quadratic becomes

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)\|^2.$$

So DDPM training is still ELBO-based, but in practice it becomes an MSE on the injected noise.

Deriving $q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$: product of Gaussians

Using Bayes and the Markov structure,

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) q(\mathbf{x}_{t-1} \mid \mathbf{x}_0).$$

Now

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \propto \exp\left(-\frac{1}{2\beta_t} \|\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1}\|^2\right),$$

and

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0, (1 - \bar{\alpha}_{t-1}) I\right).$$

So

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \propto \exp\left(-\frac{1}{2\beta_t} \|\mathbf{x}_t - \sqrt{\alpha_t} \mathbf{x}_{t-1}\|^2 - \frac{1}{2(1 - \bar{\alpha}_{t-1})} \|\mathbf{x}_{t-1} - \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_0\|^2\right).$$

Conclusion: the posterior must also be Gaussian in $\mathbf{x}_{t-1}$.

Completing the square

Expand the exponent and collect the terms in x_{t-1} . After completing the square,

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\left(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I\right),$$

where

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t,$$

and

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t.$$

Interpretation

Given the noisy state x_t , the clean image x_0 tells us exactly how to correct one step backward.

Where this posterior enters the ELBO

The DDPM ELBO contains the terms

$$\sum_{t=2}^T \text{KL}(q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0) \parallel p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)).$$

Parameterize the learned reverse kernel as

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mu_{\theta}(\mathbf{x}_t, t), \Sigma_t).$$

If we fix the reverse variance, each KL reduces to a quadratic penalty on the mean:

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_{\theta}(\mathbf{x}_t, t)\|^2 \right] + \text{const.}$$

So training reduces to matching the exact Gaussian posterior mean.

Rewrite the posterior mean in terms of the injected noise

Using the forward reparameterization

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I),$$

the exact posterior mean can be rewritten as

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon} \right).$$

So if we parameterize the model by predicting noise,

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right),$$

then the KL term becomes proportional to

$$\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)\|^2.$$

Key practical message

Mean prediction, denoising, and noise prediction are all equivalent parameterizations of the same local Gaussian reverse step.

The simplified DDPM loss and the bridge to SDEs

After dropping timestep-dependent constants, we obtain the familiar training objective

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2.$$

So the DDPM picture is:

- forward process: add tiny Gaussian noise,
- reverse process: remove tiny Gaussian noise,
- training: learn the local denoising direction.

Why the SDE view is natural

When the time steps become infinitesimal, this sequence of local Gaussian denoising problems becomes a **continuous reverse-time stochastic process**. That is exactly the SDE formulation.

discrete Gaussian reverse steps $\implies$ continuous reverse-time stochastic flow

Minimal SDE background

A time-inhomogeneous Itô SDE has the form

$$dx_t = f(x_t, t) dt + g(t) dw_t,$$

where w_t is Brownian motion.

- $f(x, t)$ is the **drift**.
- $g(t)$ controls the **diffusion strength**.
- Over a tiny interval Δt ,

$$x_{t+\Delta t} \approx x_t + f(x_t, t)\Delta t + g(t)\sqrt{\Delta t}z, \quad z \sim \mathcal{N}(0, I).$$

Interpretation: drift transports mass, diffusion spreads it.

VP SDE: the continuous-time analogue of DDPM

The variance-preserving SDE is

$$dx_t = -\frac{1}{2}\beta(t)x_t dt + \sqrt{\beta(t)} dw_t.$$

- The drift contracts toward the origin.
- The diffusion injects Gaussian randomness.
- This is the continuous-time limit of the DDPM forward chain.

Its solution has the form

$$x_t = s(t)x_0 + \sigma(t)\epsilon, \quad s(t) = \exp\left(-\frac{1}{2} \int_0^t \beta(u) du\right), \quad \sigma(t)^2 = 1 - s(t)^2,$$

with $\epsilon \sim \mathcal{N}(0, I)$.

The forward perturbation kernel in continuous time

For the VP SDE,

$$q_t(x_t | x_0) = \mathcal{N}(s(t)x_0, \sigma(t)^2 I), \quad x_t = s(t)x_0 + \sigma(t)\epsilon.$$

This is the exact continuous-time counterpart of the discrete formula

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

Important conceptual bridge

The discrete DDPM variables $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1 - \bar{\alpha}_t}$ become smooth functions $s(t)$ and $\sigma(t)$.

Density evolution and the Fokker–Planck equation

If $p_t(x)$ is the density of x_t , then

$$\partial_t p_t(x) = -\nabla \cdot (f(x, t)p_t(x)) + \frac{1}{2}g(t)^2 \Delta p_t(x).$$

- First term: transport by drift.
- Second term: smoothing by diffusion.
- The forward SDE is chosen so that p_T becomes close to a simple Gaussian.

Goal of generation: learn how to reverse the density evolution from p_T back to p_0 .

Why the score appears naturally

The central object is the time-dependent score field

$$\nabla_x \log p_t(x).$$

- It points toward higher-density regions of p_t .
- At large noise, it captures coarse global geometry.
- At small noise, it captures fine local structure.

Core message

Once we know the score field for every noise level, we can write reverse-time dynamics that generate data.

Reverse-time SDE theorem

For the forward SDE

$$dx = f(x, t) dt + g(t) dw,$$

the time-reversed dynamics are

$$dx = \left[f(x, t) - g(t)^2 \nabla_x \log p_t(x) \right] dt + g(t) d\bar{w},$$

with time now running from T down to 0.

- The drift is corrected by the score term.
- Without that correction, the process would keep diffusing outward.

For VP specifically,

$$dx = \left[-\frac{1}{2}\beta(t)x - \beta(t)\nabla_x \log p_t(x) \right] dt + \sqrt{\beta(t)} d\bar{w}.$$

The learning problem: approximate the score

We want a neural network $s_\theta(\mathbf{x}, t)$ such that

$$s_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

The ideal objective would be

$$\mathbb{E}_{p_t(\mathbf{x})} \|s_\theta(\mathbf{x}, t) - \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\|^2,$$

but this is not directly usable because the true score of the noisy marginal is unknown.

Fix: use the known perturbation kernel $q_t(\mathbf{x}_t | \mathbf{x}_0)$ and derive a denoising score matching objective.

Derivation step 1: score of the Gaussian perturbation kernel

For

$$q_t(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(s(t)\mathbf{x}_0, \sigma(t)^2 I),$$

its log-density is

$$\log q_t(\mathbf{x}_t \mid \mathbf{x}_0) = C - \frac{1}{2\sigma(t)^2} \|\mathbf{x}_t - s(t)\mathbf{x}_0\|^2.$$

Differentiate with respect to $\mathbf{x}_t$:

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{\mathbf{x}_t - s(t)\mathbf{x}_0}{\sigma(t)^2}.$$

Since $\mathbf{x}_t = s(t)\mathbf{x}_0 + \sigma(t)\epsilon$,

$$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t \mid \mathbf{x}_0) = -\frac{\epsilon}{\sigma(t)}.$$

This is the tractable target we will regress against.

Derivation step 2: posterior averaging identity

The noisy marginal is

$$p_t(x_t) = \int q_t(x_t | x_0) p_{\text{data}}(x_0) dx_0.$$

Differentiate under the integral and divide by $p_t(x_t)$:

$$\nabla_{x_t} \log p_t(x_t) = \mathbb{E} [\nabla_{x_t} \log q_t(x_t | x_0) | x_t].$$

Meaning

The score of the noisy marginal equals the conditional expectation of the score of the known corruption kernel.

- So if we regress on the kernel score, we are learning the true marginal score in expectation.
- This is the key consistency statement behind denoising score matching.

Derivation step 3: the continuous-time score loss

Choose t from some distribution on $[0, T]$. Then the denoising score matching objective is

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_t \lambda(t) \mathbb{E}_{\mathbf{x}_0 \sim p_{\text{data}}, \mathbf{x}_t \sim q_t(\cdot | \mathbf{x}_0)} \|s_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t | \mathbf{x}_0)\|^2.$$

Substitute the Gaussian target:

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \lambda(t) \left\| s_{\theta}(\mathbf{x}_t, t) + \frac{\epsilon}{\sigma(t)} \right\|^2.$$

- $\lambda(t)$ reweights different noise levels.
- The target is explicit because the perturbation kernel is explicit.

From score prediction to noise prediction

Parameterize the score network as

$$s_{\theta}(\mathbf{x}_t, t) = -\frac{1}{\sigma(t)}\epsilon_{\theta}(\mathbf{x}_t, t).$$

Plug this into the previous objective:

$$\mathcal{L}_{\text{DSM}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \lambda(t) \frac{1}{\sigma(t)^2} \|\epsilon_{\theta}(\mathbf{x}_t, t) - \epsilon\|^2.$$

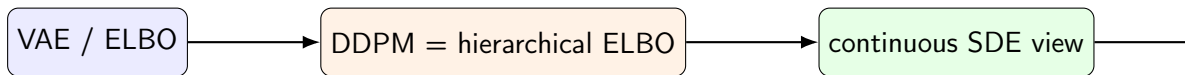
Choosing the weighting so that the prefactor is absorbed, we recover the familiar loss

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2.$$

Conclusion

The DDPM noise-prediction loss is a reparameterized form of a score approximation loss.

How the story fits together



- VAE gives the variational language.
- DDPM turns it into many Gaussian latent steps.
- SDEs provide the continuous limit.
- Score matching gives the tractable training rule.

Sampling after training

Once $s_\theta(\mathbf{x}, t)$ is learned, generate by simulating the reverse SDE

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - g(t)^2 s_\theta(\mathbf{x}, t) \right] dt + g(t) d\bar{\mathbf{w}}, \quad t : T \rightarrow 0,$$

with initial condition $\mathbf{x}_T \sim p_T$.

- Start at Gaussian noise.
- Step backward across noise levels.
- Each step uses the learned score to move toward more likely data.

A deterministic alternative is the probability flow ODE:

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2} g(t)^2 s_\theta(\mathbf{x}, t) \right] dt.$$

Conditional generation: what changes?

Suppose we want to sample from $p(x | y)$ where y is a condition:

- a class label,
- a text prompt,
- a mask or low-resolution image,
- side information or control signals.

Then the reverse dynamics should use the **conditional score**

$$\nabla_x \log p_t(x | y).$$

Bayes decomposition

$$\nabla_x \log p_t(x | y) = \nabla_x \log p_t(x) + \nabla_x \log p_t(y | x).$$

This says: unconditional structure plus condition-specific steering.

Two practical routes to conditional sampling

1. Direct conditioning in the denoiser

$$\epsilon_{\theta}(x_t, t, y) \quad \text{or} \quad s_{\theta}(x_t, t, y).$$

Use label embeddings, cross-attention, conditioning stacks, or control modules.

2. **Guidance** Combine conditional and unconditional predictions. For classifier-free guidance,

$$\hat{\epsilon} = \epsilon_{\theta}(x_t, t, \emptyset) + w \left(\epsilon_{\theta}(x_t, t, y) - \epsilon_{\theta}(x_t, t, \emptyset) \right).$$

Equivalently in score form,

$$\hat{s} = s_{\theta}(x_t, t, \emptyset) + w \left(s_{\theta}(x_t, t, y) - s_{\theta}(x_t, t, \emptyset) \right).$$

Effect: increasing w improves condition fidelity but can reduce diversity.

Why latent diffusion models were introduced

Pixel-space diffusion works well, but it is expensive.

- Images are high-dimensional.
- Reverse diffusion needs many neural evaluations.
- Most semantic structure lies on a lower-dimensional manifold.

Latent diffusion idea:

- 1 Train an autoencoder E, D so that $z = E(x)$ is a compressed latent representation.
- 2 Run diffusion in latent space on z instead of pixel space.
- 3 Decode the final latent sample with D .

Result: much cheaper diffusion with strong perceptual quality.

Interpretation: what did we gain by going latent?

- The autoencoder removes pixel-level redundancy.
- Diffusion can focus on semantic and structural variation.
- Conditioning becomes easier to inject through a latent transformer or U-Net.
- Sampling is faster because the denoiser runs on a smaller tensor.

Conceptual relation to the lecture so far:

VAE-style compression + diffusion / score-based generation in latent space.

This is another reason the VAE viewpoint is a natural starting point.

Why finance is a natural application area

Financial data are often

- high-dimensional,
- noisy,
- multi-scale in time,
- conditional on regimes, events, or macro variables,
- constrained by dependence structures and tail behavior.

Diffusion models are attractive because they can model rich distributions and generate full scenarios rather than only point forecasts.

Application 1: scenario generation for risk management

Goal: generate realistic joint scenarios for returns, volatilities, spreads, or curves.

- Produce samples from a learned distribution over future paths.
- Capture nonlinear dependence beyond Gaussian copulas.
- Stress-test portfolios under generated market scenarios.

Typical conditions could include

- market regime,
- macro variables,
- central-bank states,
- realized volatility levels.

Output: full-path distributions useful for VaR, expected shortfall, and what-if simulation.

Application 2: synthetic financial time series and data augmentation

Diffusion models can generate realistic samples for

- limit-order book states,
- intraday returns,
- volatility surfaces,
- yield curves,
- panel data across assets.

Why useful?

- augment scarce or proprietary datasets,
- anonymize sensitive data while preserving structure,
- improve downstream forecasting or classification models,
- test trading or execution systems in simulation.

Application 3: conditional generation for pricing and stress testing

A conditional model can generate paths under constraints such as

- a target initial curve,
- a macro scenario,
- an asset class or sector label,
- a stress event.

Examples:

- generate equity paths conditioned on a volatility regime,
- generate yield-curve evolutions conditioned on policy rates,
- generate option-surface scenarios conditioned on spot and realized variance.

This is exactly where conditional diffusion and classifier-free guidance become practically valuable.